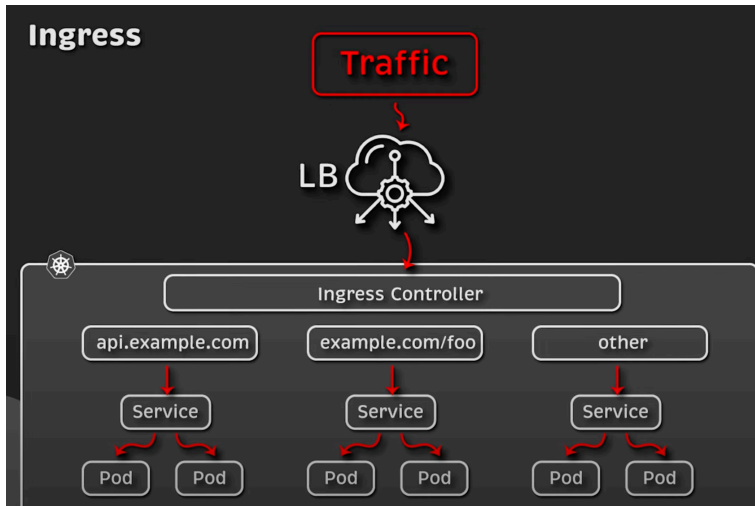


14. INGRESS controller is most powerful one. It needs only one LB and acts a router for different services operated by subdomains etc. It is also the most complicated one. It is not a service, it is a controller service. **See Docs**



15. `kubectl get nodes --show-labels`

It will show all Nodes configs. Now say I am deploying this

```
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 1
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.14.2
          ports:
            - containerPort: 80
      nodeSelector:
        disktype: ssd
```

It will deploy the pod only to the nodes with ssd label



There is also another type of selector: Affinity and anti-affinity. It can impose different types of constraints.

See this video : https://www.youtube.com/watch?v=rX4v_L0k4Hc

16.

17. Sample code for secret and config Management with Deployment

Secret and Configuration Management

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 1
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: nginx:1.14.2
          ports:
            - name: http
              containerPort: 8080
          env:
            - name: MY_USERNAME
              valueFrom:
                secretKeyRef:
                  name: creds
                  key: user
            - name: MY_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: creds
                  key: password
          volumeMounts:
            - name: config
              mountPath: /etc/config
      volumes:
        - name: config
          configMap:
            name: config
```

```
apiVersion: v1
kind: Secret
metadata:
  name: creds
data:
  user: YWRtaW4= # admin
  password: ZGV2b3BzMTIz # devops123
```

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: config
data:
  dbUrl: "postgresql://staging.example.pvt:5432"
  foo: bar
```



18. HPA and VPA (Add CPU and RAM to same pod)

Scalability

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: myapp
  namespace: default
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  minReplicas: 1
  maxReplicas: 10
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 80
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 70
```

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata:
  name: myapp
spec:
  targetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: myapp
  updatePolicy:
    updateMode: "Off"
  resourcePolicy:
    containerPolicies:
      - containerName: '*'
        controlledResources: ["cpu", "memory"]
```



19. In production multiple control planes are in action routed via an LB

